MODUL VI
INTEGRASI CONTROLLER DAN PENGUJIAN KODE

I. Tujuan Praktikum

1. Mahasiswa mampu mengimplementasikan komponen Controller sebagai jembatan yang menghubungkan antara View dan Model.
2. Mahasiswa mampu menerapkan konsep Event-Driven Programming untuk merespons interaksi pengguna pada antarmuka grafis (GUI).
3. Mahasiswa memahami dasar pengujian perangkat lunak (Software Testing) dengan menulis Unit Test menggunakan kerangka kerja pytest

II. Alat dan Bahan

1. PC/Laptop
2. Code Editor (Visual Studio Code)
3. Python 3.x
4. Laragon
5. Library pytest dan pytest-cov

III. Landasan Teori

Dalam pola Model-View-Controller (MVC), Controller bertindak sebagai koordinator atau pengatur lalu lintas data. Ketika pengguna berinteraksi dengan antarmuka (misalnya menekan tombol "Simpan" di View), Controller akan menangkap peristiwa (event) tersebut, mengambil data masukan dari View, mengirimkannya ke Model untuk disimpan ke basis data, dan akhirnya memerintahkan View untuk memperbarui tampilannya (seperti menyegarkan isi tabel). Controller memastikan View dan Model tetap terpisah secara independen. View tidak pernah menulis kueri SQL secara langsung, dan Model tidak pernah berurusan dengan elemen visual GUI.

Pada pengembangan aplikasi web berbahasa PHP, kita menggunakan index.php sebagai titik eksekusi utama yang memanggil Controller. Pada aplikasi desktop Python, pendekatan yang identik digunakan dengan membuat sebuah file main.py di root direktori proyek. File ini bertanggung jawab untuk menginisialisasi Controller lalu menyalakan putaran utama aplikasi (mainloop).

Pengujian perangkat lunak (Software Testing) adalah proses memverifikasi apakah kode berfungsi sesuai spesifikasi. Unit Testing adalah tingkat pengujian paling dasar, di mana

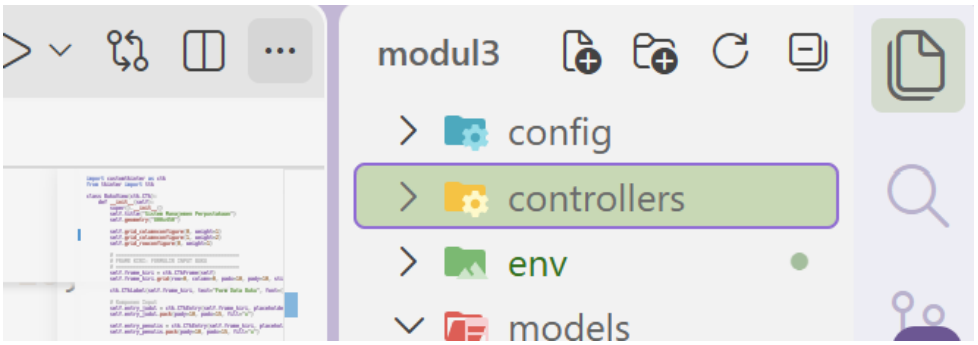
kita menguji fungsi/metode terkecil secara terisolasi. Dalam ekosistem Python, pytest adalah kerangka kerja (framework) pengujian yang sangat populer karena sintaksnya yang ringkas. Pengujian dilakukan untuk memastikan logika di Model atau Controller bebas dari bug (cacat logika) sebelum aplikasi didistribusikan.

IV. Prosedur Kerja

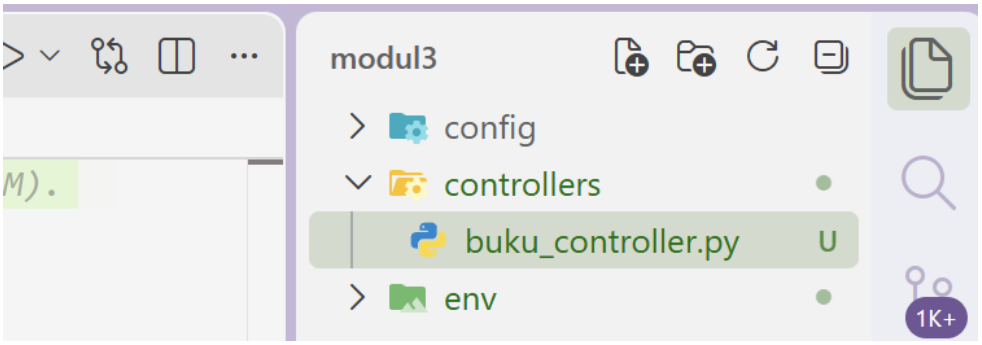
1. Instalasi Pustaka Pengujian
    a. Buka folder proyek di VS Code dan pastikan Virtual Environment (env) aktif.
    b. Ketik perintah berikut di terminal untuk menginstal pustaka testing:pip install pytest pytest-cov

```
(env) (base) PS D:\asdos\rpl\kode program\modul3> pip install pytest pytest-cov
Collecting pytest
  Downloading pytest-9.1.1-py3-none-any.whl.metadata (7.6 kB)
Collecting pytest-cov
  Downloading pytest_cov-7.1.0-py3-none-any.whl.metadata (32 kB)
```

2. Membuat Struktur Folder Controller
    a. Buat folder baru di dalam direktori proyek dengan nama controllers.



    b. Di dalam folder controllers, buat file baru bernama buku_controller.py.



3. Implementasi Controller Buku

    Tulis kode berikut di dalam file buku_controller.py. Kode ini berfungsi untuk mengikat tombol di View ke operasi basis data di Model.

main.py | database.py | buku_model.py | test_model.py | buku_view.py M | buku_controller.py U

controllers > buku_controller.py > ...

```python
import tkinter.messagebox as messagebox
from models.buku_model import BukuModel
from views.buku_view import BukuView

class BukuController:
    def __init__(self):
        # Inisialisasi Model dan View
        self.model = BukuModel()
        self.view = BukuView()

        # Binding (mengikat) tombol di View dengan fungsi di Controller
        self.view.btn_simpan.configure(command=self.simpan_data)

        # Memuat data ke tabel saat aplikasi pertama kali dibuka
        self.muat_data_tabel()

    def muat_data_tabel(self):
        # Mengosongkan isi tabel di View terlebih dahulu
        for item in self.view.tabel.get_children():
            self.view.tabel.delete(item)

        # Mengambil data dari Model (database)
        data_buku = self.model.get_all_buku()

        # Memasukkan data baru ke dalam tabel View
        for buku in data_buku:
            self.view.tabel.insert("", "end", values=(buku['id_buku'], buku['judul'], buku['penulis'], buku['tahun_terbit']))

    def simpan_data(self):
        # Mengambil teks dari kotak input di View
        judul = self.view.entry_judul.get()
        penulis = self.view.entry_penulis.get()
        tahun = self.view.entry_tahun.get()

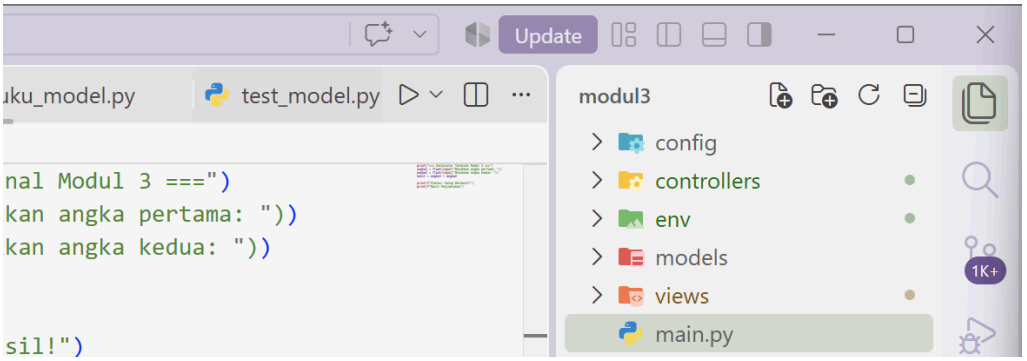
        # Validasi Form Kosong
        if not judul or not penulis or not tahun:
            messagebox.showwarning("Peringatan", "Semua kolom form harus diisi!")
            return

        # Mengirim data ke Model untuk disimpan
        berhasil = self.model.create_buku(judul, penulis, tahun)

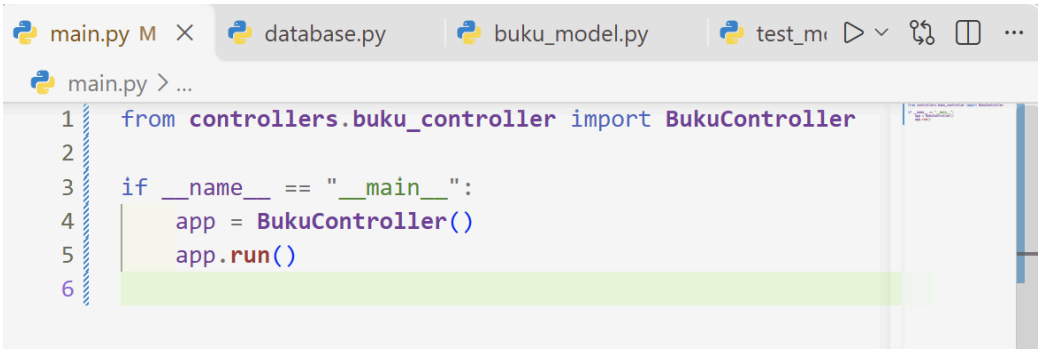
        if berhasil:
            messagebox.showinfo("Sukses", "Data buku berhasil disimpan!")
            # Bersihkan form input
            self.view.entry_judul.delete(0, 'end')
            self.view.entry_penulis.delete(0, 'end')
            self.view.entry_tahun.delete(0, 'end')
            # Muat ulang tabel agar data terbaru muncul
            self.muat_data_tabel()
        else:
            messagebox.showerror("Error", "Terjadi kesalahan saat menyimpan data.")

    # Fungsi untuk menyalakan aplikasi
    def run(self):
        self.view.mainloop()
```

4. Membuat Titik Masuk Aplikasi (main.py)
    a. Di luar semua folder (models, views, controllers), buat file main.py (berfungsi sama seperti index.php pada Modul II).



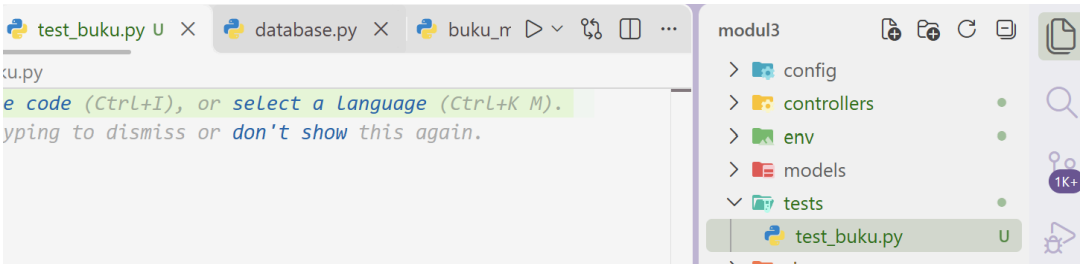
b. Isi dengan kode berikut:



c. Jalankan aplikasi dengan mengetik python main.py di terminal. Silakan uji coba mengetik data di form dan tekan tombol "Simpan Data". Jika MVC berhasil, data akan tersimpan ke MySQL dan otomatis muncul di tabel GUI.

5. Pembuatan Unit Test

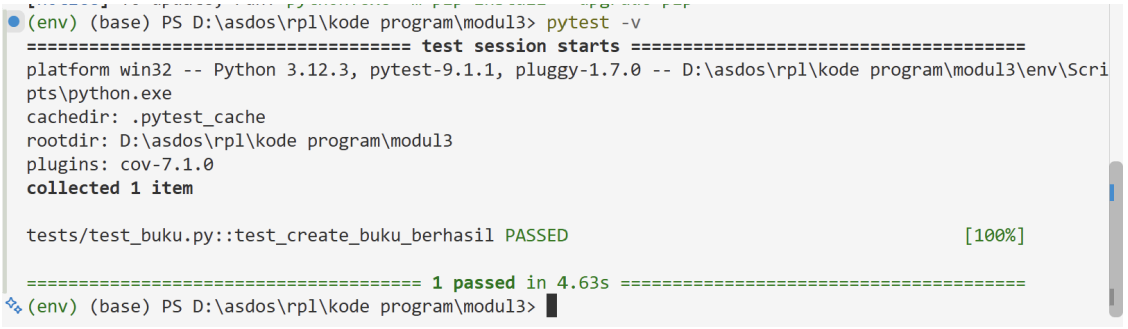
a. Buat folder baru bernama tests di dalam direktori proyek dan buat file bernama test_buku.py.



b. Tulis kode testing dasar menggunakan teknik mocking:



c. Jalankan pengujian di terminal dengan perintah: pytest -v.



6. Menyimpan Perubahan ke Repositori Git
    a. Cek status proyek: git status
    b. Tambahkan ke tahapan Git: git add .
    c. Simpan riwayat: git commit -m "feat: integrasi BukuController, file main.py, dan penambahan unit testing"
    d. Unggah ke GitHub: git push origin main

V. Hasil dan Pembahasan

1. Tampilan GUI saat berhasil menyimpan data
2. Tampilan terminal setelah testing
3. Tampilan repositori github

TUGAS

1. Pada BukuController, implementasikan logika untuk dua tombol yang Anda tambahkan di Tugas Praktikum Modul 5, yaitu tombol "Hapus Data" dan "Perbarui Data". Petunjuk: Ambil data id_buku dari baris tabel yang diklik oleh pengguna, lalu teruskan ke fungsi hapus/update di BukuModel.
2. Buat file anggota_controller.py. Implementasikan logika yang sama untuk mengikat form dan tabel Anggota yang telah Anda buat di Modul 5, ke kelas AnggotaModel yang Anda buat di Modul 4.
3. Buat satu fungsi pengujian baru (Unit Test) di file test_buku.py untuk menguji fungsi get_all_buku() menggunakan pytest. Pastikan hasil test berstatus PASSED!